

2.1 Sequential Models and RNNs

A hidden state that reads the sequence one token at a time, with shared weights.

These notes match the lecture slides. Use **(slides)** on the course hub for the deck.

An MLP sees a vector and forgets. Language is a **sequence**: the meaning of *not* depends on what came before. A recurrent net keeps a **state** and updates it at every token.

This week is the historical path to transformers. You still need it: LSTMs show up in speech and time series, and the failure mode (a chain of Jacobians) is why Week 3 exists.

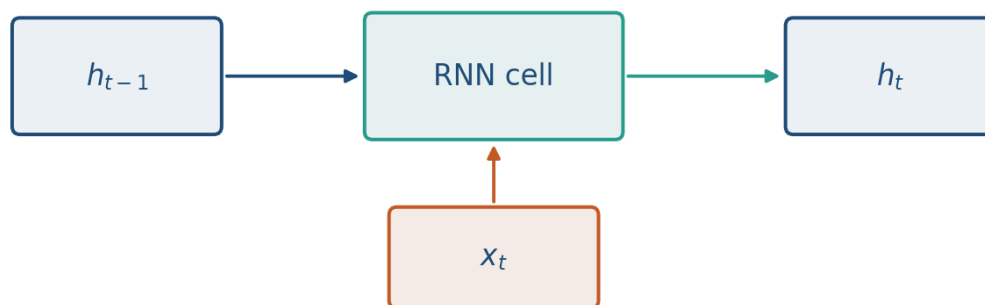
1. The cell

At step t the cell takes

- the new token embedding x_t , and
- the previous hidden state h_{t-1} .

It emits h_t , which is supposed to hold **both**. That vector is the memory.

Same weights at every step: input plus previous state



The same weights are used at every step. That is weight sharing along time, the sequence analogue of a CNN's shared filter.

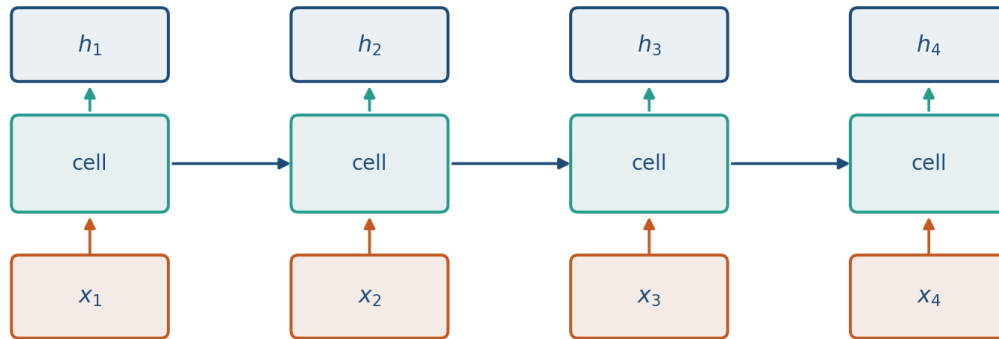
Between two independent sequences (two reviews in a batch) you **reset** $h_0 = 0$. The loop lives *inside* one sequence.

2. Unrolled in time

Write the loop as a chain. Three tokens, one set of weights:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) .$$

Unrolled in time. The chain is why long-range gradients struggle.



A CNN looks at a **fixed** neighborhood, in parallel. An RNN looks at **everything so far**, in order, and cannot fully parallelize across time. That is the accuracy / speed tradeoff.

3. What the state has to carry

If the first word is the subject of a verb twenty tokens later, h_t must still contain that subject. The cell is a lossy compressor: every step overwrites. Week **2.2** is what happens to the gradient on that long path. Week **2.3** adds gates so the cell can choose what to keep.

4. Practice

1. Unroll three tokens on paper with $d = 2$, $h_0 = 0$. Which matrices are reused?
2. Why reset the state between two unrelated documents in a batch?
3. Name one thing an RNN can mix that a kernel of size 5 cannot, in a single layer.